TASK 1

1).

```
int secondLargestUnique(vector<int>& arr) {
    int largest = INT_MIN;
    int secondLargest = INT_MIN;

    for (int i = 0; i < arr.size(); i++) {
        if (arr[i] > largest) {
            secondLargest = largest;
            largest = arr[i];
        }
        else if (arr[i] < largest && arr[i] > secondLargest) {
            secondLargest = arr[i];
        }
    }

    return secondLargest == INT_MIN ?-1:secondLargest;
}
```

2).

```
void rotateRight(vector<int>& nums, int k) {
    int n=nums.size();
    k=k%n;
    reverse(nums.begin(), nums.end());
    reverse(nums.begin(), nums.begin() + k);
    reverse(nums.begin() + k, nums.end());
}
```

3).

```
bool isAnagram(string s1, string s2) {
    string a='';
    string b='';
    for (char ch:s1) {
        if (ch != ' ')
            a+=tolower(ch);
    }

    for (char ch:s2) {
        if (ch!=' ')
            b += tolower(ch);
    }

    unordered_map<char,int>mp;
    if(a.size() != b.size()) return false;


     for(int i=0; i< a.size(); i++){
        mp[a[i]]++;
    }
    for(int i=0; i<b.size(); i++){
        mp[b[i]]--;
```

```
    }
    for(auto it=mp.begin(); it!=mp.end(); it++){
        if(it->second!=0){
            return false;
        }
    }
    return true;
}
```

4)

```
int lengthOfLongestSubstring(string s) {
    unordered_map<char,int>mp();
    int l=0;
    int r=0;
    int maximum=0;
    while(r<s.size()){
        if(mp[s[r]]==0){
            mp[s[r]]++;
           maximum=max(maximum,r-l+1);
            r++;
         }
        else{
            mp[s[l]]--;
            l++;

        }

    }
    return maximum;



}
```

5)

```
SELECT d.dept_name,
       e.name,
       e.salary
From Employees e
JOIN Departments d
    ON e.dept_id = d.dept_id
Where e.salary = (
    SELECT MAX(salary)
    From Employees
    Where dept_id = e.dept_id
);
```

6)

7)

```
async function getUserPosts() {
  try {
    const response = await fetch(
      "https://jsonplaceholder.typicode.com/posts?userId=3"
    );
```

```
    const posts = await response.json();

    for (const post of posts) {
      console.log(post.title.toUpperCase());
    }

    console.log("Total posts:", posts.length);
  } catch (err) {
    console.log("Something went wrong:", err.message);
  }
}

getUserPosts();
```

8)

```
async function getWeather() {
  const city = "London";

  const weatherCodes = {
    0: "Clear sky",
    1: "Mainly clear",
    2: "Partly cloudy",
    3: "Overcast",
    45: "Fog",
    61: "Light rain",
    80: "Rain showers",
    95: "Thunderstorm"
  };

  try {
    const response = await fetch(
      "https://api.open-meteo.com/v1/forecast?
latitude=51.5072&longitude=-0.1276&current_weather=true"
    );

    const data = await response.json();

    const temperature = data.current_weather.temperature;
    const weatherCode = data.current_weather.weathercode;

    console.log(`City       : ${city}`);
    console.log(`Temperature: ${temperature}°C`);
    console.log(
      `Condition  : ${weatherCodes[weatherCode] || "Unknown"}`
    );
  } catch (error) {
    console.log("Failed to fetch weather data:", error.message);
  }
}

getWeather();
```